

DOCUMENTO MAESTRO DEL PROYECTO

Sistema Integrador de Reservaciones para Community Tours Sian Ka'an

Dato	Información
Versión	2.2 - Checkpoint 1: Backend base de Reservaciones
Fecha de actualización	24 de agosto de 2026
Estado	Checkpoint 1 completado; módulo base de reservaciones implementado, probado y refactorizado
Avance global estimado	Aproximadamente 40 %
Tecnologías	Electron, HTML/CSS/JavaScript, Node.js + Express, PostgreSQL

1. Descripción general

El Sistema Integrador de Reservaciones tiene como objetivo centralizar y administrar en una sola aplicación de escritorio las reservaciones de Community Tours Sian Ka'an. El sistema busca sustituir la dispersión de información entre plataformas y hojas de cálculo, facilitar la operación diaria y proporcionar trazabilidad de los cambios.

Las fuentes contempladas son FareHarbor, GetYourGuide, WhatsApp Business y reservaciones externas. El sistema no administra pagos; los importes asociados a las reservaciones se conservan como información operativa.

2. Alcance funcional

- Inicio de sesión y control de acceso por roles.
- Dashboard con resumen general de reservaciones.
- Registro, consulta, modificación y cancelación lógica de reservaciones.
- Búsqueda por código, cliente y fecha.
- Historial de reservaciones.
- Bitácora de cambios y trazabilidad.
- Generación del Daily por fecha, vista previa, exportación a PDF e impresión.
- Notificaciones operativas.
- Integraciones futuras con FareHarbor, GetYourGuide y WhatsApp Business, sujetas a APIs, permisos y credenciales.

3. Roles del sistema

3.1 Administrador / Reservaciones

- Visualizar, crear, modificar y cancelar reservaciones.
- Consultar historial y bitácora.
- Generar, exportar e imprimir el Daily.
- Consultar notificaciones.

3.2 Usuario de Consulta

- Iniciar sesión.
- Consultar y buscar reservaciones.
- Consultar históricos.
- Sin permisos para modificar información.

La autorización deberá comprobarse en el backend; ocultar controles en la interfaz no sustituye la validación de permisos.

4. Arquitectura técnica

Se mantiene una arquitectura por capas con API REST. Electron funcionará como contenedor de escritorio; la interfaz utilizará HTML5, CSS3 y JavaScript, tomando Google Stitch/Figma como referencia visual. El backend utiliza Node.js + Express y PostgreSQL como gestor de base de datos.

Electron → Frontend HTML/CSS/JS → HTTP/REST → Node.js + Express → PostgreSQL

El frontend nunca se conecta directamente a PostgreSQL. Toda operación de datos debe pasar por el backend.

5. Tecnologías y entorno

Componente	Tecnología / herramienta	Estado
Base de datos	PostgreSQL + pgAdmin 4	Validado
Backend	Node.js + Express	Funcionando
Driver PostgreSQL	pg	Instalado y probado
Configuración	dotenv	Instalado
Desktop	Electron	Instalado; integración pendiente
Frontend	HTML5 + CSS3 + JavaScript	Pendiente de implementación
Diseño UI	Google Stitch / Figma	Referencia visual definida
Versionado	Git	Activo; checkpoints mediante commits
Asistente de desarrollo	Codex + AGENTS.md	En uso incremental

6. Modelo de datos

La base de desarrollo se denomina sian_kaan_reservaciones. El modelo conserva siete tablas principales y seis claves foráneas, de acuerdo con el DER validado.

- roles
- usuarios
- tours
- paises
- plataformas
- reservaciones
- bitacora

Relación	Cardinalidad
roles → usuarios	1:N
usuarios → bitacora	1:N
tours → reservaciones	1:N
paises → reservaciones	1:N
plataformas → reservaciones	1:N
reservaciones → bitacora	1:N

6.1 Tabla reservaciones

Campos consolidados: id_reservacion, codigo, fecha, id_tour, id_pais, id_plataforma, nombre_cliente, habitacion, pax, ninos, pickup_place, pickup_time, precio_total, deposito, saldo, tipo_cambio, metodo_pago, estado, fecha_registro y ultima_actualizacion. Las claves numéricas utilizan GENERATED ALWAYS AS IDENTITY; los importes usan NUMERIC(10,2) y las fechas de registro usan TIMESTAMP.

7. Validación de la base de datos

- Base sian_kaan_reservaciones creada en PostgreSQL.
- Siete tablas y seis claves foráneas verificadas.
- Pruebas de INSERT, SELECT, UPDATE y JOIN realizadas correctamente.
- Integridad referencial comprobada mediante rechazo de una FK inexistente.
- Registro de acciones en bitacora probado a nivel de base de datos.
- Los datos de prueba son temporales y pueden eliminarse posteriormente.

- database/schema.sql preparado y database/seeds.sql creado para datos iniciales de desarrollo.

8. Backend y conexión PostgreSQL

Node.js, Express, pg y dotenv están configurados. La conexión a PostgreSQL está centralizada en backend/config/database.js y usa variables de entorno. La ruta temporal GET /api/test-db continúa disponible como comprobación de conexión.

El backend funcional de reservaciones ya sigue la separación Route → Controller → Service → PostgreSQL.

9. Checkpoint 1 - Backend base de Reservaciones

Este checkpoint representa la primera sección funcional estable del sistema. Las operaciones básicas del módulo de reservaciones fueron implementadas y probadas contra PostgreSQL.

Método y endpoint	Función	Estado
GET /api/reservaciones	Listar reservaciones por registro reciente	Completado y probado
GET /api/reservaciones/:id	Consultar reservación por ID	Completado y probado
POST /api/reservaciones	Crear reservación	Completado y probado
PATCH /api/reservaciones/:id	Modificar parcialmente	Completado y probado
PATCH /api/reservaciones/:id/cancelar	Cancelar lógicamente	Completado y probado

9.1 Comportamientos validados

- Consultas SQL parametrizadas.
- Respuestas HTTP controladas para casos 200/201, 400, 404, 409 y 500 según la operación.
- Validación de identificadores, campos obligatorios y valores numéricos.
- Manejo controlado de violaciones de FK y código de reservación duplicado.
- fecha_registro y ultima_actualizacion gestionadas por el servidor/PostgreSQL.
- La cancelación no usa DELETE: conserva el registro y cambia estado a Cancelada.
- Cancelar dos veces es idempotente: la segunda operación no modifica ultima_actualizacion.
- La edición y cancelación conservan conceptualmente el estado anterior para facilitar la futura bitácora.

10. Refactor del módulo de reservaciones

Antes de continuar con nuevas funciones se realizó un refactor de mantenibilidad. El controller había crecido a aproximadamente 567 líneas y concentraba validaciones, conversiones y manejo repetido de errores. El comportamiento público de los endpoints se mantuvo sin cambios y se repitieron las pruebas del módulo.

Archivo / capa	Responsabilidad actual
routes/reservaciones.routes.js	Definición de rutas del módulo.
controllers/reservaciones.controller.js	HTTP, coordinación, respuestas y llamadas a validadores/servicios.
validators/reservaciones.validator.js	Reglas reutilizables, conversiones y normalización de datos.
services/reservaciones.service.js	Consultas SQL parametrizadas y operaciones de datos.
utils/dbErrors.js	Traducción centralizada de errores PostgreSQL conocidos.
config/database.js	Pool/conexión centralizada a PostgreSQL.

Resultado: separación real de responsabilidades, menor duplicación y una base más preparada para búsquedas, autenticación, roles y bitácora.

11. Estructura actual relevante del repositorio

```
sistema-integrador-reservaciones/
  backend/
    config/database.js
    controllers/reservaciones.controller.js
```

```

routes/reservaciones.routes.js
services/reservaciones.service.js
validators/reservaciones.validator.js
utils/dbErrors.js
server.js
database/
  schema.sql
  seeds.sql
docs/
electron/
frontend/
.env / .env.example / .gitignore
AGENTS.md
README.md
package.json / package-lock.json

```

12. Git, seguridad y forma de trabajo

- Repositorio local en rama master.
- Los cambios funcionales se guardan mediante commits después de plan, implementación y pruebas.
- .env permanece excluido mediante .gitignore; .env.example no contiene contraseña real.
- AGENTS.md define stack, arquitectura, seguridad y límites para Codex.
- Flujo adoptado: plan con Codex → revisión → implementación → pruebas → git status/diff → commit.
- No se permite que Codex cambie silenciosamente modelo de datos, tecnologías, roles, seguridad o integraciones.

13. Estado actual del proyecto

Área	Estado
Requerimientos funcionales y no funcionales	Completados / validados
Arquitectura técnica	Completada
DER y modelo de datos	Completado
PostgreSQL y relaciones	Implementados y probados
Infraestructura Node/Express/pg	Funcionando
Backend base de reservaciones	Checkpoint 1 completado
Refactor/mantenibilidad reservaciones	Completado
Búsquedas y filtros	Siguiente bloque
Autenticación / login	Pendiente
Roles y permisos backend	Pendiente
Bitácora integrada	Pendiente
Historial	Pendiente
Frontend funcional	Pendiente
Electron integrado	Pendiente
Daily / PDF / impresión	Pendiente
Notificaciones	Pendiente
Integraciones externas	Pendientes
Pruebas integrales / instalador	Pendientes

Avance global estimado: aproximadamente 40 %. Este porcentaje es orientativo y pondera análisis, arquitectura, base de datos, backend, frontend, integraciones, pruebas y despliegue; no es un promedio simple de tareas.

14. Trabajo pendiente y ruta de continuación

Etapas	Contenido
Bloque 2	Búsquedas y filtros: código, nombre de cliente y fecha; posteriormente evaluar filtros por tour, plataforma y estado.
Bloque 3	Autenticación / Login y decisión técnica JWT vs sesiones.

Bloque 4	Roles y permisos en backend para Reservaciones y Usuario de Consulta.
Bloque 5	Bitácora real: usuario, fecha/hora, acción y cambios; integrar modificación/cancelación mediante transacciones.
Bloque 6	Historial de reservaciones.
Bloque 7	Frontend HTML/CSS/JavaScript basado en los diseños existentes.
Bloque 8	Integración del frontend con Electron.
Bloque 9	Daily por fecha, vista previa, PDF e impresión.
Bloque 10	Notificaciones operativas.
Bloque 11	Integraciones con FareHarbor, GetYourGuide y WhatsApp Business según APIs/permisos disponibles.
Bloque 12	Pruebas integrales, empaquetado Electron, instalador, validación con la empresa y entrega final.

15. Decisiones técnicas abiertas

- JWT o sesiones para autenticación.
- Librería y estrategia de generación del Daily en PDF.
- Reglas definitivas de estados y si una reservación Cancelada podrá volver a modificarse.
- Formato definitivo del código único de reservación.
- Estrategia concreta de integración/sincronización con FareHarbor.
- Estrategia concreta de integración/sincronización con GetYourGuide.
- Integración técnica definitiva con WhatsApp Business.
- Modelo definitivo para observaciones del Daily y notificaciones, ya que el esquema actual no agrega tablas/campos sin aprobación.
- Estrategia final de empaquetado y distribución de Electron.

16. Próximo paso inmediato

Iniciar el Bloque 2: búsquedas y filtros del módulo de reservaciones. El primer objetivo será extender la consulta de reservaciones para permitir búsquedas operativas por código, nombre del cliente y fecha, manteniendo la arquitectura y las pruebas actuales.

17. Conclusión

El proyecto superó la etapa de infraestructura inicial y alcanzó su primer checkpoint funcional. PostgreSQL, Node.js, Express y la comunicación backend-base de datos están validados; el módulo base de reservaciones ya permite listar, consultar, crear, modificar y cancelar sin eliminación física. El refactor realizado reduce la complejidad del controller y deja una estructura más mantenible. El siguiente ciclo se concentrará en búsquedas/filtros y, después, autenticación, roles, bitácora e interfaz.